Customer-Support Response Generation with a GRU Encoder–Decoder Report

Wifek Chabir · M2 IASD · NLP Course Project

1. Introduction

The task is to generate a customer-support agent's reply from a customer's message. Automating first replies can reduce response time and agent workload. Formally, given an input sequence x, the model generates a sequence y token by token, maximizing $p(y \mid x) = \prod_{t=1}^{T} p\,(y_t \mid y_{<t}, x)$. We implement and train from scratch (PyTorch) a GRU encoder–decoder and compare it with the same model extended with attention. We use GRU layers because they are simpler and faster than LSTM, with similar performance on this task

2. Related concepts

An encoder–decoder reads the input into a hidden representation, and a decoder generates the output one token at a time. In the baseline the whole input is compressed into one vector (the final encoder state), a possible bottleneck. Attention lets the decoder weight all encoder states at each step and build a context vector. Training uses teacher forcing; generation is autoregressive.

3. Dataset

Bitext Customer Support LLM Chatbot Training Dataset (Hugging Face): 26,872 synthetic pairs (11 categories, 27 intents). Input = customer message (`instruction`), target = agent reply (`response`). Split 80/10/10, stratified by intent: 21,497 train / 2,687 validation / 2,688 test. Inputs are short (median 9 subword tokens, 99th percentile 16); responses are long (median 105, 95th percentile 244).

4. Preprocessing

Text is lowercased and whitespace-normalized (line breaks are removed). Placeholders such as `{{Order Number}}` become single tokens `<order_number>` so the model can copy them. A shared SentencePiece BPE tokenizer (vocabulary 4,000) is trained on the training split only, to avoid leakage. Subwords split rare or misspelled words, so only 0.14% of training tokens are `<UNK>`. Special tokens: `<PAD>`=0, `<UNK>`=1, `<SOS>`=2, `<EOS>`=3; targets are `<SOS>` + tokens + `<EOS>`. Inputs are truncated to 24 tokens (nothing is lost in practice) and targets to 256 tokens (95.6% of responses stay complete). Batches are padded with `<PAD>`, which is ignored by the loss, skipped by the encoder (packed sequences) and masked in attention. Token embeddings have 128 dimensions.

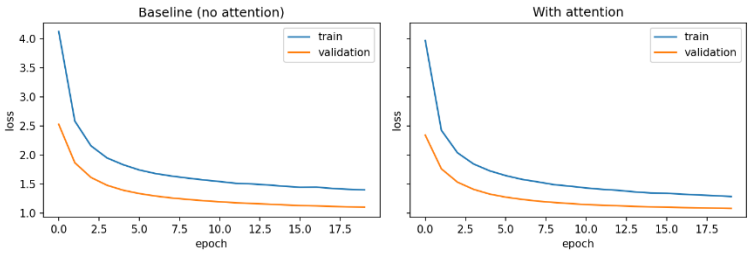
5. Proposed architecture

- **Encoder:** Embedding (4000×128) → 1-layer unidirectional GRU (hidden size 256). Its final hidden state initializes the decoder.
- **Decoder:** at each step, the previous token → Embedding → GRU (256) → Linear (256→4000) → softmax over the vocabulary.
- **Attention variant:** additive attention scores each encoder state against the decoder's previous hidden state, score $= v^T \tanh(W_e h_i + W_d s_{t-1})$. A softmax over non-padding positions gives the weights, and the weighted sum is the context vector. It is concatenated to the GRU input and to the GRU output before the linear layer.
- **Parameters:** 2,644,896 (baseline) and 3,997,088 (attention).

6. Training

Token-level cross-entropy (ignoring `<PAD>`), Adam (learning rate 1e-3), batch size 64, 20 epochs, gradient clipping 1.0, dropout 0.2, seed 42. Teacher-forcing ratio 0.8: at each step the true previous token is fed with probability 0.8, otherwise the model's own prediction. Validation uses ratio 1.0, and the checkpoint with the lowest validation loss is kept. Both models share exactly the same settings; the only difference is attention.

7. Experimental results



Both validation curves decrease until epoch 20 and never rise, so there is no overfitting; the training loss is higher than the validation loss because it is measured with dropout and partial teacher forcing. Test perplexity matches validation perplexity.

Generation uses greedy decoding, stopping at <EOS> or 254 tokens; references are truncated to the same length. Perplexity and token accuracy diagnose next-token prediction; BLEU and ROUGE compare complete generated replies with the reference. Exact match is not used because many different replies are valid.

8. Generated examples

Successes: For "i need assistance changing the address", the attention model reproduces the reference almost exactly (ROUGE-L 0.86) and asks for the current and the new address, whereas the baseline paraphrases and omits the current address (0.53). For the order-status request it keeps the key phrases of the reference (0.89 vs. 0.68).

Failures:

(1) *Repetition loop:* for "assistance submitting feedback for ur services", greedy decoding with attention repeats "areas where we excel and" until the length limit (ROUGE-L 0.11); such loops appear in about 4.0% of attention outputs versus 0.9% for the baseline.

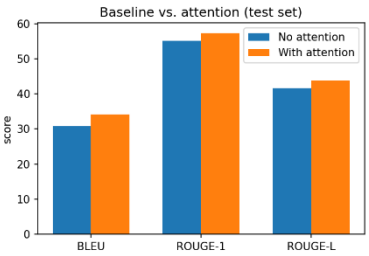
(2) *Generic deflection:* for "are there early termination fees?" and "how long reimbursements take", the references give direct information, but both models ask for account or order details or invent a procedure (0.13–0.18).

(3) *Language slips*, e.g. the baseline's "the long reimbursements for reimbursements".

By intent and length: Attention's ROUGE-L is lowest for review, edit_account and change_shipping_address (about 0.36) and highest for switch_account and cancel_order (0.56). The longest responses are not the hardest (0.49 vs. about 0.42 for mid-length ones), probably because long replies share more common phrases with the generated text.

9. Discussion

Attention improves every metric, more on generated text (BLEU +3.3, about +11%) than on loss (about −2%), likely because it keeps the reply tied to the input words. It is better on 60% of test examples, so the gain is an average effect, not a guarantee. The gain is modest because inputs are very short, so one vector already carries most of the information. The price is 1.5× the parameters and 2.2× the training time, and attention loops about four times more often, a known weakness of greedy decoding.



Failures are fluent but wrong in content: the models learn the style of support replies, not company facts, so they deflect or invent procedures. Limitations: the data is synthetic and templated (13.5% of test messages also appear in training, with different replies), so scores are optimistic compared with real chats; each model was trained once, so small differences may partly be noise; 4.4% of responses are truncated and line breaks are removed; BLEU and ROUGE use a single reference and penalize valid paraphrases; only greedy decoding was tested.

10. Conclusion

Both from-scratch models learn to write fluent, relevant support replies, and adding attention gives a consistent but moderate gain at a clear computational cost. Main lessons: a fair comparison needs identical data and settings, loss alone hides differences that BLEU and ROUGE reveal, and analyzing failures matters as much as the scores. Possible future work: a bidirectional encoder, repeated runs with different seeds, a Transformer.